What The Font: Written Evaluation

Minna Kimura-Thollander (mkimurat), Katherine Sang (ksang1), Maggie Wu (mwu27)

Github: https://github.com/artset/WhatTheFont

Introduction

Fonts are the language of design, serving several industries, including advertising, publishing, web development, public transportation, and graphic design. Given the rise in UI/UX as a field, font identification— as well as discovering similar fonts— is a relevant tool that designers can use to easily discover fonts similar to premium fonts, or alternatives to widely used fonts.

We reimplemented DeepFont, a font recognition model authored by researchers at Adobe, Snapchat, and the University of Illinois at Urbana-Champaign. The model works such that when given an image of a font, it can identify the font as well as find the top five most similar fonts to the given font.

Preprocessing / Data

We used font images from the AdobeVFR dataset, which includes real data (pictures taken from real artifacts that display English text in a specific font) and synthetic data (computer generated English text in a specific font).

Our Data Distribution

	Synthetic	Real
Autoencoder	150 000	1 920 000
DeepFont Train	750 000	0
DeepFont Test	150 000	60 000

Original vs. Our Implementation

	Ours	Original
Font classes	150	2 383
Image size	96 x 96	105 x 105
Total images	3 000 000	37 500 000

Fig 1. Charts displaying our data distribution. Notice that we had to cut down on total images, because we simply did not have enough computational power to process 37.5 million images. As part of this process, we hand-selected 150 fonts.

The unlabeled and labeled real data came in the form of raw images that could be downloaded directly. However, the labeled synthetic data was provided in the form of a BCF file due to the original authors' concerns about Adobe copyright. Working with a BCF file proved to be challenging due to the lack of online resources, but we were lucky to be provided with the original code that the authors used to package their PNG files into a single BCF file. We were able to reverse engineer this file to unpack the BCF into actual images, successfully extracting 2.3 million raw images of synthetic data.

Once we had all of our raw images, we preprocessed them by applying noise, blur, perspective rotation, and shading to each. This was done to prevent the model from overfitting, while allowing it to train on potentially "messier" images found in the real world. For instance, designers often manipulate original typefaces by distorting text and rotating certain letters. Thus, applying these alterations were essential to allow our model to identify fonts given variable conditions in the real world.

After applying these alterations, we randomly selected ten 96 × 96 pixel patches of each font image that function as inputs to the model. We want to note that the original model used 105 × 105 pixel patches—we reduced the size to 96 so that performing convolution and deconvolution would lead to outputs of the same size.

Model Architecture

Our implementation includes two models: a Convolutional Autoencoder and the actual DeepFont model.

Convolutional Autoencoder

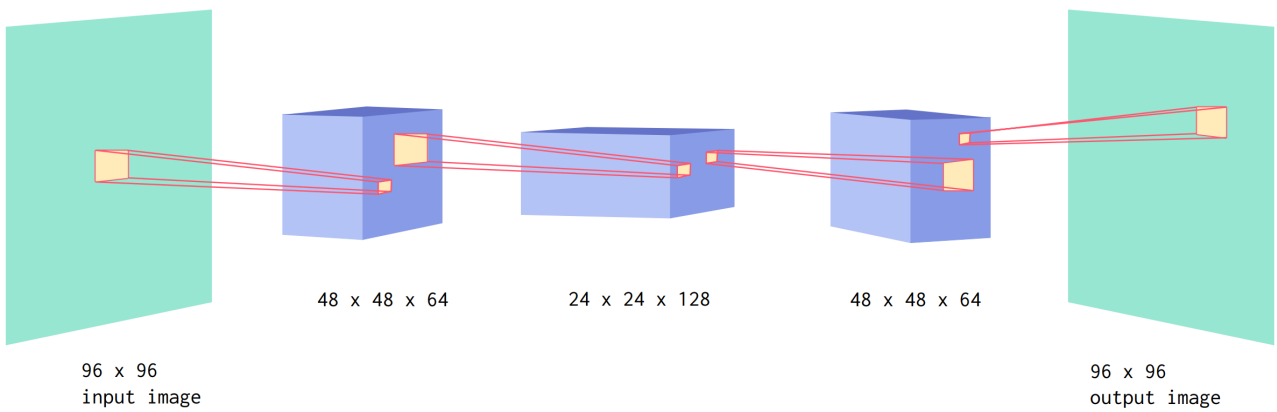


Fig 2. Convolutional autoencoder architecture. Model diagram created in Adobe Illustrator by us!

The convolutional autoencoder is an unsupervised network used to learn lower-level, abstract features of different fonts. Given that there we had fewer real labeled training examples, the autoencoder is how our final model becomes exposed to a large number of real world samples without needing labels.

HYPERPARAMETERS

batch size	128
activation	leaky relu
learning rate	0.01 (Adam)
epochs	5

Deepfont Model (Original and Modified)

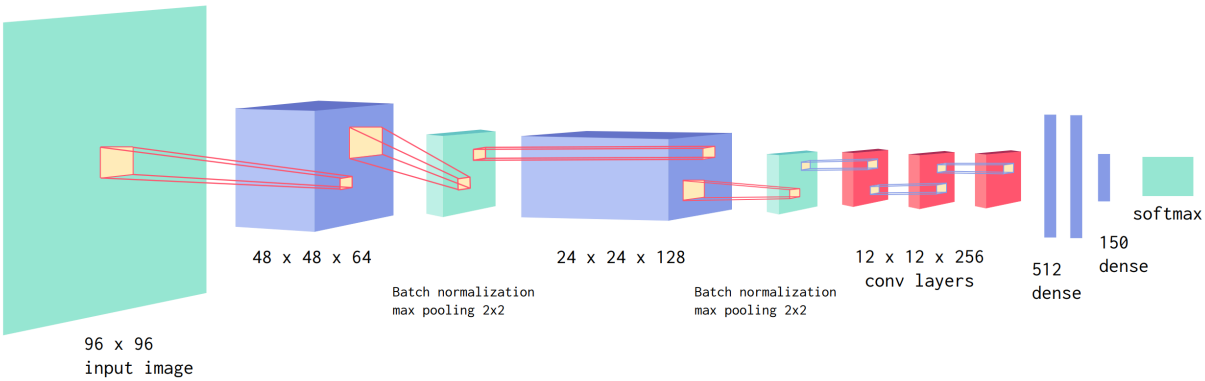


Fig 3. Deepfont model architecture. Model diagram created in Adobe Illustrator by us!

The DeepFont model is a supervised CNN. The weights of the first two convolutional layers are imported from the autoencoder. These weights are frozen and not trained any further within the DeepFont model to prevent new data from altering any information gained from the autoencoder. This is followed by three convolutional layers, which are then flattened and passed through a series of dense layers. Finally, we softmax to obtain the probability distribution over the 150 fonts that we chose to analyze.

We also tried a second approach, which was a suggestion from Professor Ritchie. In this method, the number of filters in the last three convolution layers is increased in order to identify more font features. We then perform global pooling, where we average across the x and y dimensions, resulting in a $1\ x\ (num_filters)$ array. This operation takes each filter and averages it into a single value, which functions as our logits. Lastly, we use a size 150 dense layer with softmax activation to derive the probability distribution over the 150 fonts.

HYPERPARAMETERS

batch size	128
activation	leaky relu
learning rate	0.01 (Adam)
epochs	10

Results

CONVOLUTIONAL AUTOENCODER LOSS

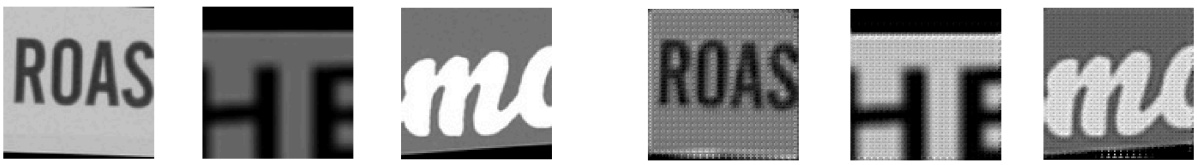
Epoch	1	2	3	4	5
Loss	26.610722	0.477306	0.2397055	0.070799	0.0067023

Fig 4. Our convolutional autoencoder performed fairly well; after the first epoch, the loss decreased significantly.

CONVOLUTIONAL AUTOENCODER OUTPUT

We believe that our autoencoder model was fairly successful, since it retained most of the edges of each typeface, though the contrast differed in the output images.

Below we display some outputs of the model:



Original inputs Autoencoder outputs

Fig 5. Results of the autoencoder model. Interestingly, the output of the “ROAS” image is dimmer with less contrast, while the output of the “HE” image is brighter with more contrast.

DEEP FONT QUANTITATIVE RESULTS

To quantitatively evaluate our model, we computed accuracy for the model, where accuracy was defined as the correct font label being in the **top five** most similar fonts recommended by the model.

Epoch	2	4	6	8	10
Accuracy for original	0.8887	0.8955	0.8963	0.8967	0.8969
Accuracy for modified	0.9213	0.9267	0.9291	0.9288	0.9300

Fig 6. The accuracy for both our models was fairly high. Since our modified version performed (marginally) better than our original implementation, we decided to do more quantitative testing on the modified version.

For each font, we kept a net score of how many times it was classified correctly and incorrectly, and graphed the top 3 best and worst performing fonts. This was done on the modified model.

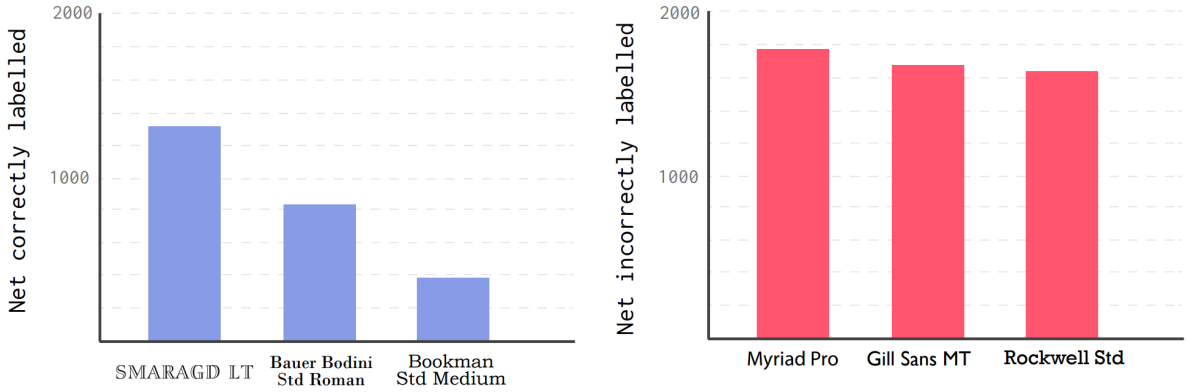


Fig 7. Best performing fonts (left) and worst performing fonts (right).

Looking at the best performing fonts, they tend to be quite stylized with serifs, with varying thickness. In contrast, the top two worst performing fonts were sans serifs of fixed thicknesses. Thus, we hypothesize that our model tends to have more difficulty learning less stylized fonts like Myriad Pro and Gill Sans MT, because they may have fewer distinguishing characteristics.

DEEP FONT CASE STUDY RESULTS

When we tested our model on cursive fonts, the model performed very well. The font of the image below is "Leander Script Pro," and you can see that the model was able to identify it as one of the top five fonts. We believe that our model worked best with such stylized fonts because there are distinct characteristics of cursive fonts that set them apart.

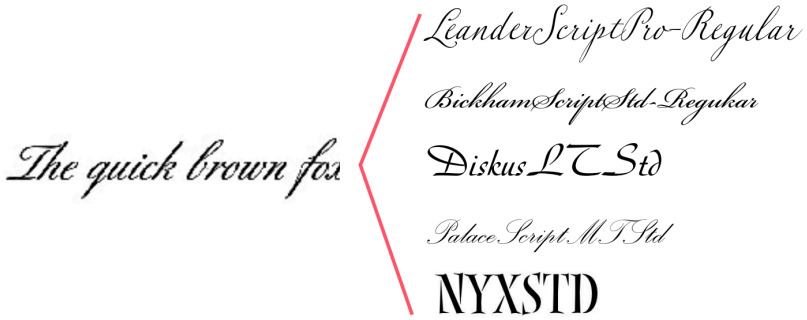


Fig 8. In the image above, the left is our input image and the right displays the recommended fonts, in order of most probable.

However, when we tested our model on the label on Professor Ritchie's door, we received poor results. The original font for Professor Ritchie's door is Calibri Bold, which is not in our dataset. Still, this does not provide an excuse for our model recommending such dissimilar fonts, and further confirmed our suspicion that our model performs poorly on sans serif fonts.

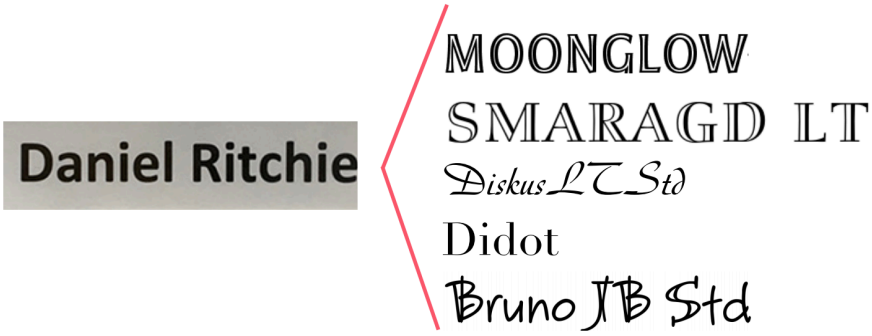


Fig 9. Testing the label on Daniel Ritchie's office door gave poor results. :(

Challenges

One of the biggest challenges was the sheer amount of data we had to handle. Not only did the data set have 2.3 million synthetic training images, but it had an additional 200,000 real training images and 5,000 real test images. Because 15 crops need to be generated for each image, this means that we had a total of 37.5 million images to process. Though we tried to use all of the data at the beginning of the project, we quickly discovered through OOM errors that we did not have the computational space or power we needed. To remedy this, we chose to cut down to 150 fonts, which resulted in our total image count being 3 million. Specifically, we chose 150 fonts that were in the "standard" style, rather than **bold** or *italics*.

Additionally, the original model from the paper was quite poorly documented. For example, the paper provides no information about the kernel size of the convolutional layers, the loss function used, or the number of epochs to train the model. There is also little information about how much blur and noise to apply to the images, and how much rotational and translational freedom the images should have. Lastly, the model diagram is not annotated coherently— hence why we made our own model diagrams!

Other challenges that we encountered along the way included needing to shuffle, store, and read large amounts of data, relabeling font classes (because we cut down to 150 fonts), and restructuring our model to work with a smaller subset of the data.

Discussion & Future Work

There were many limitations in our implementation, including not being able to use the full set of data, as well as not having all of the information needed to replicate the paper's model. The original authors tested their model on different sets of images, including different alterations to their synthetic data, in combination with different amounts of real data. Further exploration could be done to find which combinations of data can yield the best results.

Another issue with our implementation is the abnormally high accuracy. After the first epoch, our model was already able to classify 89% of the images correctly. However, this high accuracy is incompatible with the case study performed on Daniel Ritchie's door label, which makes us believe that our model is

particularly weak at classifying sans serif fonts. In the future, we would want to improve the performance on sans serif fonts by increasing the amount of data we have for them and experimenting with hyperparameters.

In regards to data, an opportunity to improve the AdobeVFR dataset could be to add more up-to-date, modern web fonts, while adding more digital examples in the real training data. Pervasive typefaces on the web such as Proxima Nova, Lato, and Roboto are not present in the data set. Additionally, since most of the images are of signs and newspapers, the training data is slightly outdated given the importance of fonts in the digital realm.

Lastly, a limitation of the original paper is that the model assumes that the image passed in contains only one font. Given that having multiple fonts in a single image is quite common, future work is necessary to build a model that can classify an image that contains numerous fonts.

Reflection

Overall, this was a challenging project, with the difficulties heavily lying in the preprocessing step, and understanding how to use and configure GCP instances and buckets to handle the amount of data we had. As a group, we learned a ton about the issues surrounding preprocessing that we didn't get to experience through class projects. Thus, we felt the project was very worthwhile, and that it challenged us to think more carefully about balancing data storage and model performance.

Although our model did not perform amazingly on font classification tests, it's interesting to see that it has a pattern to its weaknesses, such as difficulties classifying sans serif typefaces. It was also an interesting experience looking at other papers in an academic setting, an experience that some of us did not have before. Not having all the information we needed to perfectly reimplement the original authors' model gave us more leeway to experiment and try out different model architectures and hyperparameters, which gave us a more realistic experience of research work than class assignments could.

Working through the entire pipeline of the process, from unpacking and preprocessing data to analyzing results, was a great experience for us, and definitely gave us a lot more insight into how to approach deep learning problems on our own in the future. Running into problems caused us to actively seek out other resources, which exposed us to tools such as Google Cloud Buckets and hdf5 files that we now know how to use in future projects. Overall, our team greatly enjoyed the whole process, and feel that we have definitely gained a lot from the project! 😊🎉